

SYNOPSIS

1. Project Title

Music App – MERN Stack Music Player Web Application

2. Statement of the Problem

With the rapid growth of digital media, users expect music platforms that let them browse, play, organize, and manage songs easily through a web interface. Most existing solutions either require heavy clients (like dedicated desktop/mobile apps) or rely on third-party platforms with limited customization. There is a need for a **simple, responsive, user-centric music player web app** that provides user authentication, playlist management, and playback control through modern web technologies.

The problem addressed by this project is the **lack of open-source, full-stack web applications** that combine a user account system with dynamic music browsing and playback features in a seamless, responsive UI. This project aims to bridge that gap by building a functional music app using the MERN stack (MongoDB, Express, React, Node.js), integrating third-party APIs (like Spotify) for real music data, and supporting common user actions like signing up, logging in, creating playlists, and playing tracks.

3. Why This Topic Was Chosen

This project was chosen because:

- **Real-world relevance:** Music streaming and playback web applications are widely used and expected to be intuitive, responsive, and scalable.
- **Learning value:** It leverages modern full-stack development tools and techniques — both frontend and backend — which are essential skills for MERN developers.
- **Technical challenge:** Implementing authentication, REST APIs, state management, and real-time UI interactions in a single integrated system provides valuable experience in developing complete applications.
- **Extensibility:** Once developed, the project can be expanded with features like user favourites, recommendations, or integration with additional music APIs.

The topic reinforces full-stack design, enabling students to demonstrate a working cross-platform web solution combining database, server, and frontend layers.

4. Objective and Scope of the Project

Primary Objectives

- Build a **secure user authentication** system with sign-up and log-in.
- Allow users to **browse and search music** using a real music API (e.g., Spotify).
- Provide features to **create and manage playlists**.
- Implement a **web-based music player** with controls: play, pause, skip, and volume.

- Store user data, playlists, and related metadata in a **database**.

Scope

- The application will be responsive and user-friendly on both desktop and mobile.
- It will integrate external music data (Spotify API) for track information and preview.
- Users can control playback and manage personal playlists after logging in.
- The backend will support REST APIs to interact with the frontend securely.
- Deployment will be possible on cloud hosting platforms for real usage.

This project does *not* focus on streaming full-track copyrighted music or offline downloads — its purpose is to provide a functional, legal music discovery and playback interface using API previews.

5. Methodology (Summary of the Project)

The project follows a **structured development methodology** involving requirement analysis, design, implementation, testing, and deployment.

Requirement Analysis

- Identify user needs: authentication, browsing songs, creating playlists, and controlling playback.
- Determine tech stack requirements for scalable web app development.

System Design

- **Frontend:** Built with React.js for dynamic UI rendering; Redux for state management.
- **Backend:** Node.js and Express.js to build REST APIs interacting with the frontend and database.
- **Database:** MongoDB to store user accounts, playlists, and related data.

Implementation

- Create UI components using React and Tailwind CSS.
- Develop API endpoints for user registration, login, playlist operations, and music fetch.
- Use JWT (JSON Web Tokens) for secure session management.
- Integrate Spotify API to fetch music tracks, albums, and search results.
- Connect backend and database using Mongoose as an ODM.
- Control playback with HTML5 audio elements and React state synchronization.
- Deploy backend and frontend on cloud platforms (e.g., Render).

Integration and Testing

- Perform unit tests on critical modules (auth, playlist actions).

- End-to-end testing to verify user flows (register → browse → play → logout).
- Responsive design validation across screen sizes.
- Debug and optimize API calls and UI performance for smooth playback.

Deployment

- Backend and frontend deployed using free or low-cost cloud services.
- Ensuring environment variables (DB URI, JWT secrets) are securely configured.
- Final testing on hosted URLs for real-world performance.

By following incremental builds and regular testing, the final system delivers a seamless music browsing and playback experience.

6. Hardware and Software to Be Used

Hardware

- PC or Laptop capable of running Node.js and development tools.
- Internet connection for API access and deployment.

Software

Category	Tools & Technologies
OS	Windows / Linux / macOS
Frontend	React.js, Redux, Tailwind CSS
Backend	Node.js, Express.js
Database	MongoDB (Atlas or local)
Authentication	JWT (JSON Web Tokens)
API Requests	Axios
API Source	Spotify Web API
Hosting	Render / similar cloud
Development Tools VS Code, Postman, Git/GitHub	

This combination supports rapid development, easy testing, and scalable deployment.

7. Testing Technologies Used

To ensure reliability and correctness of features, the following approaches are used:

- **Manual Functional Testing:** Verifying user actions such as sign up, sign in, playlist creation, and playing tracks.

- **Integration Tests:** Ensuring frontend components correctly call backend APIs and handle responses.
- **Unit Tests (Optional):** Using Jest or similar tools to test authentication logic and utility functions.
- **Cross-Browser Testing:** Confirm consistent performance across major browsers (Chrome, Firefox, Edge).
- **Responsive Design Testing:** Ensuring UI scales properly on desktops and mobile devices.

Any bugs discovered are logged and resolved before final submission.

8. Project Contribution / Outcome

This Music App project delivers the following:

- **Functional full-stack music web application** with user accounts and playlist features.
- **Learning and demonstration tool:** Practical implementation of MERN stack development.
- **Reusable template** for future projects or extensions (e.g., adding favorites, history, offline support).
- **Improved UI and UX** through real music browsing with playback controls.

The project strengthens understanding of web architecture, API integration, security, and responsive design. It provides a working solution that fills the need for a simple and scalable music player on the web.